

Prompt Engineering & Structured Output (Anthropic Docs)

Anthropic (2024–2026). “Prompt engineering overview” and “Tool use / structured output.” Claude Developer Documentation, docs.anthropic.com. Accessed August 2026.

ASSIGNED READING

Prompt-engineering overview + the technique pages

~20 min

Structured output — skim only the schema part of the tool-use page

~5 min

» ~25 min docs • this summary + definitions + reasoning notes ~22 min

What to ignore on that page. Anthropic documents structured output inside its tool-use pages, so that one page covers two different things. You want the idea that you can declare a schema the model must satisfy. You do NOT need the tool-definition, tool-choice and multi-turn orchestration material — that is function calling, and it belongs to Class 4, where you will build an agent with a tool. In Class 3 we get our structure the simpler way: we ask for a strict JSON shape in the prompt and then verify it in code. Skim for the concept, skip the plumbing.

Note on currency. These are living documentation pages — wording and examples change. The techniques below are stable; if a detail on the site differs from this summary, the site is authoritative. Examples use Claude, but the techniques transfer to any modern instruction-tuned model.

KEY TAKEAWAY

You can't change a model's weights, but you can change what you put in front of it — and that is most of the practical craft. Anthropic's guidance is a ladder of techniques, roughly in the order you should try them: be clear and direct; then show examples; then let the model think step by step; then structure the prompt (XML tags, a system/role prompt, prefilling); and finally chain or decompose big tasks. Separately, structured output — forcing the model to answer in a fixed shape such as JSON — is how you make its output machine-usable and checkable.

WHY IT MATTERS FOR THIS COURSE

This is the practical backbone of C2 (“Prompting craft”) and the coding build. Every technique in the capsule maps to a page here. Structured output is the specific technique the Week-3 build depends on: you'll prompt a model to extract fields into strict JSON (JavaScript Object Notation) so that a program — and an eval harness — can check the result. “Programming in words” is exactly this ladder.

Feeds Class 3: C2 (every prompting technique) • the coding build (structured output → eval) • Assignment 1 part (iii).

THE WORDS, DEFINED — read this first if the vocabulary is new

These pages assume the vocabulary. If any of these terms is new, three minutes here will save you twenty later. Fuller technical treatments of the reasoning techniques are in Case Pre-read 1 (Med-PaLM 2).

- **Token** — the unit a model actually reads and writes: a word, a piece of a word, or a punctuation mark. Everything is priced and measured in tokens, so “make the prompt longer” always means “pay more.”
- **Prompt** — what the model actually receives for one request. Technically it is not a single string: the API takes a list of messages, each tagged with a role — system, user, and assistant (the model's own earlier replies) — and flattens them into one token sequence before the model sees anything. So your instruction, your examples, the document you pasted and every prior turn all land in the same context window, and all of them are billed as input tokens.
- **System prompt** — the message carrying the system role. Three things make it different from an ordinary user message. First, position and persistence: it sits first and is re-sent unchanged on every request, because the API is stateless — the model remembers nothing between calls, and “the whole conversation” is really your code resending the system message plus every previous turn each time. Nothing is stored on the provider's side, so a long system prompt is paid for again on every single call. Second, weighting: instruction-tuned models are trained to treat system-role content as more authoritative than user-role content, so a rule placed there survives contradiction better than the same rule buried in the user message. Note this is a learned tendency, not an enforced guarantee — which is exactly why prompt injection works at all. Third, caching: because it is stable across calls, it is the natural cacheable prefix, and providers bill cached input at a lower rate.

Where to put what. Durable things go in the system message — the role, the tone, standing rules, the output contract, safety constraints. Per-request things go in the user message — this document, this question. The same words cost the same in either place, but they hold up differently when the model is pulled in two directions.

In your Capsule 2 notebook. This is literal rather than conceptual: `generate()` builds `[{"role": "system", ...}, {"role": "user", ...}]`, `SYSTEM` holds “You are a careful data-extraction assistant for a Singapore bank. You never invent

values,” and the cost tally adds SYSTEM to the user prompt before estimating tokens — because the system message is billed on every call too.

- **Zero-shot / few-shot (multishot)** — how many worked examples you include before your real question. Zero-shot is just asking; few-shot shows the model two or three solved instances first. Usually the highest-leverage single change.
- **Chain of thought (CoT)** — asking the model to write out its reasoning step by step before giving the answer. Better on multi-step problems, and more expensive, because the reasoning is generated text you pay for.
- **XML tags** — wrapping parts of the prompt in labels such as `<context>...</context>` or `<example>...</example>`, so the model can tell your instructions apart from the data you pasted. Also your first line of defence against a document that contains instructions of its own.
- **Prefilling** — starting the model’s reply for it — opening with “{”, for instance — to force a format or skip a preamble you do not want.
- **Prompt chaining / decomposition** — splitting a complicated job into a sequence of smaller prompts, each doing one thing, instead of one prompt attempting everything. Easier to debug, and easier to see which step failed.
- **JSON** — a plain-text format for labelled data: named fields with values, rather than a paragraph of prose. It is what lets a program pick out “the loan amount” without reading the sentence around it.
- **Schema** — the declared shape of that JSON — which fields must be present, what type each one is, what values are allowed. It is the contract you check the answer against.
- **Structured output** — the general technique of making the model return data in a fixed schema instead of free text. This is the C2 build, and the reason your prompt becomes testable at all.
- **Tool use / function calling** — letting the model call a function you have defined and use the result. Some providers also use this machinery to enforce a schema, which is why it appears on the same documentation page. We do NOT do tool calling in Class 3 — it is the core of Class 4.
- **Eval set** — a fixed list of test cases with known-good answers, written before you start tuning, so that “the prompt got better” becomes a number rather than an impression. You build one in Capsule 2.

MAIN POINTS

Start simple: be clear and direct

- **Say exactly what you want.** Ambiguity is the most common cause of a bad answer. State the task, the audience, the format and any constraints explicitly — treat the model like a capable new colleague who has no context beyond your words.

Show, don’t just tell: examples

- **Few-shot / multishot.** Giving a handful of input → output examples (“shots”) is often the single highest-leverage move. It demonstrates the pattern and the exact format you want, far more reliably than describing it in prose.

Let the model think: chain of thought

- **Chain of thought (CoT).** Asking the model to reason step by step before answering improves accuracy on anything involving logic, maths or multi-step analysis. The trade-off: more tokens, so more cost and latency — a C3 theme.

Structure the prompt

- **XML tags.** Wrapping parts of the prompt in tags (e.g. `<context>...</context>`, `<example>...</example>`) helps the model tell instructions from data and keeps long prompts organised.
- **System prompts / roles.** Setting a role in the system prompt (“You are a financial-compliance analyst...”) steers tone and expertise for the whole conversation.
- **Prefilling.** You can begin the model’s reply for it (for example, opening with “{”) to force a format or skip preamble.

Break big tasks up

- **Chain prompts / decompose.** For complex work, split it into a sequence of smaller prompts, each doing one thing, rather than one giant prompt. Easier to debug and more reliable — the same “decomposition” idea the Prompt Report catalogues (Reading 4).

Structured output — the build-critical technique

- **Fix the shape of the answer.** Ask for output in a strict schema (JSON with named fields). Now the result is something code can consume directly, instead of prose you have to parse.

- **Tool use makes it reliable.** Anthropic's tool-use / JSON mode lets you define the exact schema the model must return — far more dependable than hoping free-text JSON comes back well-formed. This is what turns a prompt into a component you can test.
- **Why it enables evals.** Once output is structured, a program can check it automatically — is the field present? is the value in range? — which is precisely the “L1 assertion” layer in C2's eval half.

READING FOCUS QUESTIONS — bring your answers to class

There are no wrong answers here — these surface how you think, not what you recall. A clue sits under each question to get you started; the fuller reasoning follows after all three, so you can arrive at a view on your own.

- Pick a task you'd hand a model. Which rung of the ladder (clear instruction → examples → chain of thought → structure → decomposition) do you think it needs, and why start there?

Clue. *Diagnose before you prescribe. Write down what actually went wrong with a plain instruction first, because the failures are not interchangeable: a wrong format is a different problem from wrong reasoning, and both differ from the model doing three jobs at once, all of them badly. Then match the failure to the rung — format problems want examples or a fixed schema; reasoning problems want chain of thought; “it did everything a bit poorly” wants decomposition. Note the default this implies: start at the cheapest rung that could plausibly fix your specific failure, not the most powerful rung available.*

- Chain of thought improves quality but costs more tokens. For your task, is the extra accuracy worth the extra cost — and how would you decide?

Clue. *Notice that “is it worth it?” cannot be answered as asked — worth needs two numbers. Put rough figures on both sides: what does one wrong answer cost you (rework, a bad decision, a lost customer, a regulator), and what does the extra reasoning cost per request in output tokens and latency? Then ask the question that actually settles it: how would you know the accuracy improved at all? If you cannot measure the gain, you are buying it on faith — which is exactly why Capsule 2 makes you write the eval set before you tune the prompt.*

- Why does forcing JSON output make a model's answer testable in a way that a paragraph of prose is not?

Clue. *Ask what a computer can do automatically with each. Try writing down, in plain words, a check you could run on a paragraph of prose — then the same check on a named field. One of them needs a human, or another model, to read and judge; the other is a comparison a single line of code can make. Then push one step further: structure does not only make checking possible, it makes failure specific. You learn which field broke, rather than that the answer was somehow bad.*

HOW TO THINK ABOUT THESE — read after you have had a go

These are not model answers to be memorised — they are the line of reasoning we will pick up in class. Class time is short, so this section is written to let you settle the argument for yourself. If your view differs and you can defend it, bring it.

Q1 — Start from the failure, not from the technique.

The ladder is easy to misread as a quality ranking, where decomposition is “best” and a clear instruction is for beginners. It is not; it is a diagnostic sequence, and each rung answers a different complaint. If the model understood the job but returned it in the wrong shape, give it examples or fix the schema — reasoning will not help. If it got the shape right but skipped a step or reached a conclusion that does not follow, that is a thinking failure and chain of thought is the fix. If it produced something mediocre across the board, it is probably doing several jobs in one pass and wants decomposing into a chain of smaller prompts. Two practical consequences. First, climb from the bottom: the cheap rungs are cheap to test and often sufficient, and you learn something from each attempt. Second, change one thing at a time — if you add examples and reasoning and a schema together and the result improves, you have learned nothing about which of them earned its place, and you are now paying for all three forever.

Q2 — Deciding whether to buy reasoning.

Frame it as a purchase rather than a preference. On the cost side, chain of thought is paid in generated tokens, which are the expensive kind, plus the latency a user waits through — and it is paid on every request, not once. On the value side, the number that matters is not “how much better does it feel” but the cost of the errors it prevents: an extraction mistake that a human catches in ten seconds is cheap, the same mistake reaching a credit decision or a patient is not. So the decision rule is roughly: buy reasoning where the task is genuinely multi-step and where a wrong answer is expensive, and withhold it on easy, high-volume traffic. But the honest answer to “how would you decide?” is that you cannot decide responsibly without measurement. Run both prompts against the same eval set, compare pass-rates, and look at the token counts alongside. That comparison — one change, measured — is the whole method of Capsule 2, and Capsule 3 attaches prices to it.

Q3 — Why structure is what makes a prompt testable.

Prose can only be judged by something capable of reading it — a person, or another model acting as judge. Both are slow, expensive and themselves fallible, so you cannot run them over hundreds of cases every time you touch your prompt. A named field changes the nature of the check: “is date_of_birth present, and does it match YYYY-MM-

DD?” is a comparison a program makes instantly, identically, for free, a thousand times. That is what turns prompting from an opinion into an experiment — you can now change one thing and see a number move. Two things worth adding. First, structure makes failure diagnosable, not merely detectable: you learn that the currency field broke on non-SGD inputs, which tells you what to fix, where “the answer was bad” tells you nothing. Second, structure is what makes the output usable by the rest of a system at all — code can consume a field, it cannot consume a paragraph. This is why the coding build in Capsule 2 starts by forcing JSON: it is the hinge between a demo and something you could actually ship.

SOURCE LINKS

Prompt-engineering overview · docs.anthropic.com/en/docs/build-with-claude/prompt-engineering/overview

Structured output (inside the tool-use docs — schema section only; tool calling itself is Class 4) ·

docs.anthropic.com/en/docs/build-with-claude/tool-use/overview

Companion · The Prompt Report (Reading 4) — the academic taxonomy behind these techniques.

CAVEATS

1. This summary is not a substitute for the docs. They are living pages — read the source; it is the authority.
2. This summary was generated and fact-checked using Anthropic models against the documentation as of August 2026. Flag anything that looks wrong.